ZyntraPay

Low-Level Design Document

Detailed Class, Sequence, ER & Workflow Design

.NET 8 Microservices Architecture

Version 1.0 | 2026

Table of Contents

Table of Contents 2
1. Document Overview 4
1.1 Purpose 4
1.2 Scope 4
1.3 Design Goals 4
2. AuthService — Detailed Design 5
2.1 Class Diagram 5
2.2 AuthDB — ER Diagram 6
2.3 Registration Sequence Diagram 7
2.4 Login Sequence Diagram 8
2.5 OTP / Forgot-Reset Password Flow 8
2.6 Refresh Token Flow 9
2.7 Key Method Specifications 9
3. UserService — Detailed Design 10
3.1 Class Diagram 10
3.2 UserDB — ER Diagram 11
3.3 KYC Submission Sequence Diagram 12
4. WalletService — Detailed Design 13
4.1 Class Diagram 13
4.2 WalletDB — ER Diagram 14
4.3 Top-Up Sequence Diagram 14
4.4 Transfer Sequence Diagram 15
5. RewardsService — Detailed Design 16
5.1 Class Diagram 16
5.2 RewardsDB — ER Diagram 17
5.3 Redeem Flow Sequence Diagram 18
5.4 Award Points (Event Consumer) Flow 18
6. NotificationService — Detailed Design 19
6.1 Class Diagram 19
6.2 NotificationDB — ER Diagram 20
6.3 Notification Flow Sequence Diagram 20
7. AdminService — Detailed Design 21
7.1 Class Diagram 21
7.2 AdminDB — ER Diagram 22

7.3 Admin KYC Review Sequence Diagram 22
7.4 Dashboard Assembly Flow 23
8. Exception Handling Design 24
8.1 Global Exception Middleware 24
8.2 Exception Categories....24
9. Caching Design....24
10. Messaging Design....25
10.1 Event Publishers....25
10.2 Event Consumers 25
11. Security Design....26
11.1 JWT Claims 26
11.2 Role Checks 26
11.3 Token Forwarding in AdminService 26
12. Testing Scope 26
12.1 Unit Test Areas....26
12.2 Integration Test Areas 26
12.3 Critical Test Cases 27
13. Complete Business Workflow Diagrams....28
13.1 End-to-End User Onboarding Workflow....28
13.2 Wallet Top-Up and Reward Award Combined Flow 29

1. Document Overview

1.1 Purpose

This Low-Level Design (LLD) document provides detailed class-level design, method specifications, ER diagrams, sequence diagrams, business workflow designs, exception handling strategies, caching design, messaging design, and security design for the ZyntraPay microservices platform. It is intended for developers implementing or maintaining any service within the system.

1.2 Scope

This document covers all seven ZyntraPay services: AuthService, UserService, WalletService, RewardsService, NotificationService, AdminService, and ApiGateway. It includes class-level diagrams, ER diagrams, and detailed sequence diagrams for every major workflow.

1.3 Design Goals

- Provide unambiguous, implementable class and method specifications grounded in the existing codebase.
- Define all database tables with keys, constraints, and relationships.
- Document every business workflow as a sequence diagram.
- Establish clear exception handling, caching, and event messaging contracts.

2. AuthService — Detailed Design

2.1 Class Diagram

AuthService — Class Diagram

```
┌──────────────────────────────────────────────────────────────┐
│  AuthController                                              │
│  ──────────────────────────────────────────────────────────  │
│  + Register(RegisterDto) : IActionResult                     │
│  + RegisterAdmin(RegisterAdminDto) : IActionResult           │
│  + Login(LoginDto) : IActionResult                           │
│  + GoogleLogin(GoogleLoginDto) : IActionResult               │
│  + UpdatePhone(UpdatePhoneDto) : IActionResult [JWT]         │
│  + SendOtp(SendOtpDto) : IActionResult                       │
│  + VerifyOtp(VerifyOtpDto) : IActionResult                   │
│  + ForgotPassword(ForgotPasswordDto) : IActionResult         │
│  + ResetPassword(ResetPasswordDto) : IActionResult           │
│  + RefreshToken(RefreshTokenDto) : IActionResult             │
│  + GetAllUsers() : IActionResult [Admin]                     │
│  + ToggleUserStatus(Guid id) : IActionResult [Admin]         │
│  + GetEmailByAuthUserId(Guid authUserId) : IActionResult     │
└──────────────────────────────┬───────────────────────────────┘
                               │ uses
                               ▼
┌──────────────────────────────────────────────────────────────┐
│  IAuthService / AuthService                                  │
│  ──────────────────────────────────────────────────────────  │
│  + RegisterAsync(RegisterDto) : AuthResponseDto              │
│  + RegisterAdminAsync(RegisterAdminDto) : AuthResponseDto    │
│  + LoginAsync(LoginDto) : AuthResponseDto                    │
│  + GoogleLoginAsync(GoogleLoginDto) : AuthResponseDto        │
│  + UpdatePhoneAsync(Guid userId, string phone) : void        │
│  + SendOtpAsync(string email) : void                         │
│  + VerifyOtpAsync(VerifyOtpDto) : bool                       │
│  + ForgotPasswordAsync(string email) : void                  │
│  + ResetPasswordAsync(ResetPasswordDto) : void               │
│  + RefreshTokenAsync(string refreshToken) : AuthResponseDto  │
│  + GetAllUsersAsync() : List<UserDto>                        │
│  + ToggleUserStatusAsync(Guid id) : void                     │
│  + GetEmailByAuthUserIdAsync(Guid id) : string               │
└──────────────────────────────┬───────────────────────────────┘
                               │ uses
                               ▼
┌──────────────────────────────────────────────────────────────┐
│  IAuthRepository / AuthRepository                            │
│  ──────────────────────────────────────────────────────────  │
│  + GetByEmailAsync(string email) : User                      │
│  + GetByIdAsync(Guid id) : User                              │
```

```
│  + ExistsByEmailAsync(string email) : bool                    │
│  + ExistsByPhoneAsync(string phone) : bool                    │
│  + CreateAsync(User user) : void                              │
│  + UpdateAsync(User user) : void                              │
│  + GetAllAsync() : List<User>                                 │
└──────────────────────────┬──────────────────────────────────┘
                           │
┌──────────────────────────▼──────────────────────────────────┐
│  AuthDbContext                                                 │
│  DbSet<User> Users                                             │
└────────────────────────────────────────────────────────────┘


Supporting Classes:
  JwtTokenHelper: GenerateToken(User) → string
  JwtTokenHelper: GenerateRefreshToken() → string
  IMemoryCache: OTP storage, RefreshToken storage, ResetToken storage
  RabbitMQ Publisher: OtpRequestedEvent, WelcomeEmailRequestedEvent
```

2.2 AuthDB — ER Diagram

AuthDB — Entity Relationship

```
┌──────────────────────────────────────────────────────────┐
│  USERS                                                    │
│  ──────────────────────────────────────────────────────  │
│  Id            UNIQUEIDENTIFIER  PK  DEFAULT NEWID()      │
│  Email         NVARCHAR(150)     NOT NULL  UNIQUE         │
│  PhoneNumber   NVARCHAR(20)      NOT NULL  UNIQUE         │
│  PasswordHash  NVARCHAR(300)     NOT NULL                 │
│  Role          NVARCHAR(50)      DEFAULT 'User'           │
│  IsActive      BIT               DEFAULT 1                │
│  CreatedAt     DATETIME          DEFAULT GETDATE()        │
└──────────────────────────────────────────────────────────┘

Indexes:
  IX_Users_Email   (unique)
  IX_Users_PhoneNumber  (unique)

Logical cross-service reference:
  AuthUserId (GUID) → Users.Id is referenced in all other service DBs
```

2.3 Registration Sequence Diagram

AuthService — Registration Flow

```
Client          AuthController      AuthService        AuthRepo    Cache
RabbitMQ
  |                  |                  |                  |         |        |
  |—POST /send-otp——▶|                  |                  |         |        |
  |                  |—SendOtpAsync——▶|                  |         |        |
  |                  |                  |—Generate OTP     |         |        |
  |                  |                  |—Cache OTP(10m)——————▶  |        |
  |                  |                  |—Publish OtpRequestedEvent——————▶|
  |◀—200 OK——————————|                  |                  |         |        |
  |                  |                  |                  |         |        |
  |—POST /verify-otp▶|                  |                  |         |        |
  |                  |—VerifyOtpAsync——▶|                  |         |        |
  |                  |                  |—Get OTP from Cache——————▶        |
  |                  |                  |—Mark email verified in cache         |
  |◀—200 OK——————————|                  |                  |         |        |
  |                  |                  |                  |         |        |
  |—POST /register——▶|                  |                  |         |        |
  |                  |—RegisterAsync——▶|                  |         |        |
  |                  |                  |—Check verified email flag            |
  |                  |                  |—ExistsByEmail——▶|                  |
  |                  |                  |—ExistsByPhone——▶|                  |
  |                  |                  |—BCrypt.Hash(pwd)  |                  |
  |                  |                  |—CreateUser——————▶                  |
  |                  |                  |—Publish WelcomeEmailEvent——————————▶
  |◀—JWT + Refresh——|                  |                  |         |        |
```

2.4 Login Sequence Diagram

AuthService — Login Flow

```
Client          AuthController      AuthService        AuthRepo
  |                 |                  |                  |
  |—POST /login———▶|                  |                  |
  |                 |—LoginAsync———▶|                  |
  |                 |                  |—Normalize email   |
  |                 |                  |—GetByEmail———▶
  |                 |                  |◀—User————————|
  |                 |                  |—BCrypt.Verify()   |
  |                 |                  |—Check IsActive    |
  |                 |                  |—GenerateJWT()     |
  |                 |                  |—GenerateRefreshToken()
  |                 |                  |—Cache RefreshToken|
  |◀—{accessToken, refreshToken}————|                  |
```

2.5 OTP / Forgot-Reset Password Flow

AuthService — Forgot/Reset Password Flow

```
Client          AuthController      AuthService        AuthRepo    Cache
  |                 |                  |                  |         |
  |—POST /forgot———▶|                  |                  |         |
  |                 |—ForgotPasswordAsync▶              |         |
  |                 |                  |—GetByEmail———▶         |
  |                 |                  |—Generate reset OTP          |
  |                 |                  |—Cache reset OTP(10m)———▶|
  |◀—200 OK————————|                  |                  |         |
  |                 |                  |                  |         |
  |—POST /reset———▶|                  |                  |         |
  |                 |—ResetPasswordAsync▶              |         |
  |                 |                  |—Get reset OTP from Cache———▶
  |                 |                  |—Verify OTP       |         |
  |                 |                  |—BCrypt.Hash(newPwd)         |
  |                 |                  |—UpdateUser———▶          |
  |                 |                  |—Revoke RefreshToken in Cache  |
  |◀—200 OK————————|                  |                  |         |
```

2.6 Refresh Token Flow

AuthService — Refresh Token Flow

```
Client          AuthController      AuthService        AuthRepo    Cache
  |                 |                  |                  |         |
  |—POST /refresh——▶|                  |                  |         |
  |                 |—RefreshTokenAsync▶               |         |
  |                 |                  |—Validate RefreshToken from Cache
  |                 |                  |—GetById————————▶          |
  |                 |                  |—Check IsActive    |         |
  |                 |                  |—GenerateJWT() + new RefreshToken
  |                 |                  |—Update Cache      |         |
  |◀—{newAccessToken, newRefreshToken}——|                  |         |
```

2.7 Key Method Specifications

Method	Input	Output	Key Logic / Side Effects
RegisterAsync	RegisterDto	AuthResponseDto	Check email verified in cache. Check unique email/phone. BCrypt hash. Save user. Publish WelcomeEmailRequestedEvent.
LoginAsync	LoginDto	AuthResponseDto	Normalize email. GetByEmail. BCrypt.Verify. Check IsActive. Generate JWT + refresh token. Cache refresh token.
SendOtpAsync	string email	void	Generate 6-digit OTP. Cache for 10 min. Publish OtpRequestedEvent.
VerifyOtpAsync	VerifyOtpDto	bool	Read OTP from cache. Compare. Mark email as verified in cache.
ResetPasswordAsync	ResetPasswordDto	void	Validate reset OTP from cache. BCrypt hash new password. UpdateUser. Revoke refresh token.
RefreshTokenAsync	string token	AuthResponseDto	Validate token from cache. GetById. Check IsActive. Issue new JWT + refresh pair. Update cache.
ToggleUserStatusAsync	Guid id	void	GetById. Toggle IsActive. UpdateUser.

3. UserService — Detailed Design

3.1 Class Diagram

UserService — Class Diagram

```
┌──────────────────────────────────────────────────────────┐
│  UserController                                          │
│  + CreateProfile(CreateProfileDto) : IActionResult [JWT] │
│  + GetProfile() : IActionResult [JWT]                    │
│  + SubmitKyc(SubmitKycDto) : IActionResult [JWT]         │
│  + GetKycStatus() : IActionResult [JWT]                  │
│  + GetPendingKyc() : IActionResult [Admin JWT]           │
│  + ReviewKyc(Guid kycId, ReviewKycDto) [Admin JWT]       │
└──────────────────────────┬───────────────────────────────┘
                           │
┌──────────────────────────▼───────────────────────────────┐
│  IUserService / UserService                              │
│  + CreateProfileAsync(Guid authUserId, CreateProfileDto) │
│  + GetProfileAsync(Guid authUserId) : UserProfileDto     │
│  + SubmitKycAsync(Guid authUserId, SubmitKycDto)         │
│  + GetKycStatusAsync(Guid authUserId) : KycStatusDto     │
│  + GetPendingKycAsync() : List<KycSubmissionDto>         │
│  + ReviewKycAsync(Guid kycId, ReviewKycDto)              │
└──────────────────────────┬───────────────────────────────┘
                           │
┌──────────────────────────▼───────────────────────────────┐
│  IUserRepository / UserRepository                        │
│  + GetProfileByAuthUserIdAsync(Guid) : UserProfile       │
│  + CreateProfileAsync(UserProfile) : void                │
│  + GetKycByAuthUserIdAsync(Guid) : KycSubmission         │
│  + GetKycByIdAsync(Guid) : KycSubmission                 │
│  + CreateKycAsync(KycSubmission) : void                  │
│  + UpdateKycAsync(KycSubmission) : void                  │
│  + GetPendingKycAsync() : List<KycSubmission>            │
└──────────────────────────┬───────────────────────────────┘
                           │
┌──────────────────────────▼───────────────────────────────┐
│  UserDbContext                                           │
│  DbSet<UserProfile> UserProfiles                         │
│  DbSet<KycSubmission> KycSubmissions                     │
└──────────────────────────────────────────────────────────┘
```

3.2 UserDB — ER Diagram

UserDB — Entity Relationship

```
┌──────────────────────────────────────────────────────────┐
│  USERPROFILES                                            │
│  Id            UNIQUEIDENTIFIER  PK                      │
│  AuthUserId    UNIQUEIDENTIFIER  UNIQUE  NOT NULL        │
│  FullName      NVARCHAR(200)     NOT NULL                │
│  DateOfBirth   DATE              NOT NULL                │
│  Address       NVARCHAR(500)     NOT NULL                │
│  City          NVARCHAR(100)     NOT NULL                │
│  State         NVARCHAR(100)     NOT NULL                │
│  PinCode       NVARCHAR(20)      NOT NULL                │
│  CreatedAt     DATETIME          DEFAULT GETDATE()       │
└──────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────┐
│  KYCSUBMISSIONS                                          │
│  Id                UNIQUEIDENTIFIER  PK                  │
│  AuthUserId        UNIQUEIDENTIFIER  UNIQUE  NOT NULL    │
│  DocumentType      NVARCHAR(100)     NOT NULL            │
│  DocumentNumber    NVARCHAR(100)     NOT NULL            │
│  Status            NVARCHAR(50)      DEFAULT 'Pending'   │
│  RejectionReason   NVARCHAR(500)     NULL                │
│  SubmittedAt       DATETIME          DEFAULT GETDATE()   │
│  ReviewedAt        DATETIME          NULL                │
└──────────────────────────────────────────────────────────┘


Relationships:
  USERPROFILES.AuthUserId → (logical) AuthDB.Users.Id
  KYCSUBMISSIONS.AuthUserId → (logical) AuthDB.Users.Id
  One user → one UserProfile  (enforced via UNIQUE constraint)
  One user → one KycSubmission (enforced via UNIQUE constraint)

KYC Status Enum: Pending | Approved | Rejected
```

3.3 KYC Submission Sequence Diagram

UserService — KYC Submit & Review Flow

```
User            UserController    UserService       UserRepo
  |                 |                |                |
  |—POST /kyc———————▶|                |                |
  |                 |—SubmitKycAsync▶|                |
  |                 |  (AuthUserId from JWT)          |
  |                 |                |—GetKycByAuthUserId▶|
  |                 |                |◀—null (no duplicate check)
  |                 |                |—Create KycSubmission (Status=Pending)
  |                 |                |—CreateKycAsync———▶|
  |◀—201 Created———|                |                |
  |                 |                |                |
Admin             UserController    UserService       UserRepo
  |                 |                |                |
  |—GET /admin/kyc/pending▶          |                |
  |                 |—GetPendingKycAsync▶             |
  |                 |                |—GetPendingKycAsync▶|
  |◀—List<KycSubmissionDto>————————|                |
  |                 |                |                |
  |—PUT /admin/kyc/{id}/review▶       |                |
  |                 |—ReviewKycAsync▶|                |
  |                 |                |—GetKycByIdAsync———▶
  |                 |                |—Set Status (Approved/Rejected)
  |                 |                |—Set ReviewedAt    |
  |                 |                |—UpdateKycAsync———▶
  |◀—200 OK————————|                |                |
```

4. WalletService — Detailed Design

4.1 Class Diagram

WalletService — Class Diagram

```
┌──────────────────────────────────────────────────────────────┐
│  WalletController                                            │
│  + CreateWallet() : IActionResult [JWT]                      │
│  + GetBalance() : IActionResult [JWT]                        │
│  + TopUp(TopUpDto) : IActionResult [JWT]                     │
│  + Transfer(TransferDto) : IActionResult [JWT]               │
│  + GetTransactions() : IActionResult [JWT]                   │
│  + GetTransactionById(Guid id) : IActionResult [JWT]         │
└──────────────────────────┬───────────────────────────────────┘
                           │
┌──────────────────────────▼───────────────────────────────────┐
│  IWalletService / WalletService                              │
│  + CreateWalletAsync(Guid authUserId, string email)          │
│  + GetBalanceAsync(Guid authUserId) : WalletBalanceDto       │
│  + TopUpAsync(Guid authUserId, TopUpDto) : TransactionDto    │
│  + TransferAsync(Guid authUserId, TransferDto) : TransactionDto│
│  + GetTransactionsAsync(Guid authUserId) : List<LedgerDto>   │
│  + GetTransactionByIdAsync(Guid id) : LedgerDto              │
└──────────────────────────┬───────────────────────────────────┘
                           │
┌──────────────────────────▼───────────────────────────────────┐
│  IWalletRepository / WalletRepository                        │
│  + GetByAuthUserIdAsync(Guid) : Wallet                       │
│  + GetByIdAsync(Guid) : Wallet                               │
│  + CreateAsync(Wallet) : void                                │
│  + UpdateAsync(Wallet) : void                                │
│  + AddLedgerEntryAsync(LedgerEntry) : void                   │
│  + GetLedgerByWalletIdAsync(Guid) : List<LedgerEntry>        │
│  + GetLedgerByIdAsync(Guid) : LedgerEntry                    │
└──────────────────────────┬───────────────────────────────────┘
                           │
┌──────────────────────────▼───────────────────────────────────┐
│  WalletDbContext                                             │
│  DbSet<Wallet> Wallets                                       │
│  DbSet<LedgerEntry> LedgerEntries                            │
└──────────────────────────────────────────────────────────────┘


  IMemoryCache: Wallet balance cached per AuthUserId
  IRabbitMqPublisher: publishes WalletTopUpCompletedEvent,
WalletTransferCompletedEvent
```

4.2 WalletDB — ER Diagram

WalletDB — Entity Relationship

```
┌──────────────────────────────────────────────────────────┐
│  WALLETS                                                 │
│  Id            UNIQUEIDENTIFIER  PK  DEFAULT NEWID()     │
│  AuthUserId    UNIQUEIDENTIFIER  UNIQUE  NOT NULL        │
│  UserEmail     NVARCHAR(200)     NOT NULL                │
│  Balance       DECIMAL(18,2)     DEFAULT 0               │
│  IsActive      BIT               DEFAULT 1               │
│  CreatedAt     DATETIME          DEFAULT GETDATE()       │
└──────────────────────────┬───────────────────────────────┘
                           │ 1
                           │
                           │ N
┌──────────────────────────▼───────────────────────────────┐
│  LEDGERENTRIES                                           │
│  Id            UNIQUEIDENTIFIER  PK  DEFAULT NEWID()     │
│  WalletId      UNIQUEIDENTIFIER  FK → Wallets.Id         │
│  Type          NVARCHAR(10)      NOT NULL  (CREDIT|DEBIT)│
│  Amount        DECIMAL(18,2)     NOT NULL                │
│  Description   NVARCHAR(500)     NOT NULL                │
│  ReferenceId   NVARCHAR(200)     NULL                    │
│  CreatedAt     DATETIME          DEFAULT GETDATE()       │
└──────────────────────────────────────────────────────────┘

Indexes:
  IX_LedgerEntries_WalletId
  IX_Wallets_AuthUserId (unique)
```

4.3 Top-Up Sequence Diagram

WalletService — Top-Up Flow

```
Client      WalletController    WalletService      WalletRepo    Cache    RabbitMQ
  |               |                  |                 |           |          |
  |-POST /topup-->|                  |                 |           |          |
  |               |-TopUpAsync------>|                 |           |          |
  |               |  (AuthUserId from JWT)             |           |          |
  |               |                  |-GetByAuthUserId>|           |          |
  |               |                  |-Check IsActive  |           |          |
  |               |                  |-AddLedgerEntry(CREDIT)>|    |          |
  |               |                  |-Update Wallet.Balance->|    |          |
  |               |                  |-ClearCache(AuthUserId)------>|          |
  |               |                  |-Publish WalletTopUpCompletedEvent------>|
  |<-TransactionDto|                 |                 |           |          |
```

4.4 Transfer Sequence Diagram

WalletService — Transfer Flow

```
Client      WalletController    WalletService              WalletRepo   RabbitMQ
  |               |                  |                          |          |
  |—POST /transfer▶|                  |                          |          |
  |               |—TransferAsync——▶|                          |          |
  |               |                  |—Check sender ≠ receiver   |          |
  |               |                  |—GetSenderWallet————————▶          |
  |               |                  |—GetReceiverWallet————————▶          |
  |               |                  |—Validate both wallets active          |
  |               |                  |—Check sender balance ≥ amount         |
  |               |                  |—AddLedgerEntry(sender, DEBIT)————————▶
  |               |                  |—AddLedgerEntry(receiver, CREDIT)————▶
  |               |                  |—UpdateSenderBalance————————▶          |
  |               |                  |—UpdateReceiverBalance————▶          |
  |               |                  |—ClearSenderCache          |          |
  |               |                  |—ClearReceiverCache        |          |
  |               |                  |—Publish WalletTransferCompletedEvent——▶|
  |◀—TransactionDto|                  |                          |          |
```

5. RewardsService — Detailed Design

5.1 Class Diagram

RewardsService — Class Diagram

```
┌──────────────────────────────────────────────────────────────┐
│  RewardsController                                            │
│  + GetSummary() : IActionResult [JWT]                         │
│  + GetCatalog() : IActionResult [JWT]                         │
│  + Redeem(RedeemDto) : IActionResult [JWT]                    │
│  + GetHistory() : IActionResult [JWT]                         │
└──────────────────────────┬───────────────────────────────────┘
                           │
┌──────────────────────────▼───────────────────────────────────┐
│  IRewardsService / RewardsService                             │
│  + GetSummaryAsync(Guid authUserId) : RewardSummaryDto        │
│  + GetCatalogAsync() : List<CatalogItemDto>                   │
│  + RedeemAsync(Guid authUserId, RedeemDto) : RedemptionDto    │
│  + GetHistoryAsync(Guid authUserId) : List<RedemptionDto>     │
│  + AwardPointsAsync(Guid authUserId, decimal amount) : void   │
└──────────────────────────┬───────────────────────────────────┘
                           │
┌──────────────────────────▼───────────────────────────────────┐
│  IRewardsRepository / RewardsRepository                       │
│  + GetAccountByAuthUserIdAsync(Guid) : RewardAccount          │
│  + CreateAccountAsync(RewardAccount) : void                   │
│  + UpdateAccountAsync(RewardAccount) : void                   │
│  + GetActiveCatalogAsync() : List<RewardCatalog>              │
│  + GetCatalogItemByIdAsync(Guid) : RewardCatalog              │
│  + AddRedemptionAsync(Redemption) : void                      │
│  + GetRedemptionsByAuthUserIdAsync(Guid) : List<Redemption>   │
└──────────────────────────┬───────────────────────────────────┘
                           │
┌──────────────────────────▼───────────────────────────────────┐
│  RewardsDbContext                                             │
│  DbSet<RewardAccount> RewardAccounts                          │
│  DbSet<RewardCatalog> RewardCatalog                           │
│  DbSet<Redemption> Redemptions                                │
└──────────────────────────────────────────────────────────────┘


  IMemoryCache: Catalog cached for 5 minutes
  RabbitMQ Consumer: WalletTopUpCompletedEvent → AwardPoints
  RabbitMQ Publisher: PointsAwardedEvent


Tier Calculation Logic:
```

```
  0-999 points    → Silver
  1000-4999 pts  → Gold
  5000+  pts      → Platinum
```

5.2 RewardsDB — ER Diagram

RewardsDB — Entity Relationship

```
┌──────────────────────────────────────────────────────────┐
│  REWARDACCOUNTS                                          |
│  Id            UNIQUEIDENTIFIER  PK                      |
│  AuthUserId    UNIQUEIDENTIFIER  UNIQUE  NOT NULL        |
│  TotalPoints   INT               DEFAULT 0              |
│  Tier          NVARCHAR(50)      DEFAULT 'Silver'       |
│  CreatedAt     DATETIME          DEFAULT GETDATE()      |
└──────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────┐
│  REWARDCATALOG                                           |
│  Id            UNIQUEIDENTIFIER  PK                      |
│  Title         NVARCHAR(200)     NOT NULL                |
│  Description   NVARCHAR(500)                             |
│  PointsCost    INT               NOT NULL                |
│  Stock         INT               DEFAULT -1 (unlimited)|
│  IsActive      BIT               DEFAULT 1               |
│  CreatedAt     DATETIME          DEFAULT GETDATE()      |
└──────────────────────────┬───────────────────────────────┘
                           | referenced by
┌──────────────────────────▼───────────────────────────────┐
│  REDEMPTIONS                                             |
│  Id                UNIQUEIDENTIFIER  PK                  |
│  AuthUserId        UNIQUEIDENTIFIER  NOT NULL            |
│  RewardCatalogId   UNIQUEIDENTIFIER  FK → RewardCatalog|
│  PointsSpent       INT               NOT NULL            |
│  RedeemedAt        DATETIME          DEFAULT GETDATE()  |
└──────────────────────────────────────────────────────────┘


Seed data in migrations:
  Amazon Voucher Rs.100  |  Free Movie Ticket
  Cashback Rs.50         |  Premium Membership
```

5.3 Redeem Flow Sequence Diagram

RewardsService — Redeem Flow

```
Client      RewardsController    RewardsService        RewardsRepo   Cache
RabbitMQ
  |                |                   |                     |           |       |
  |—POST /redeem—▶|                   |                     |           |       |
  |                |—RedeemAsync———▶|                     |           |       |
  |                |                   |—GetAccountByAuthUserId▶         |       |
  |                |                   |—GetCatalogItemById——▶|           |       |
  |                |                   |—Validate IsActive + Stock        |       |
  |                |                   |—Validate user has enough points  |       |
  |                |                   |—Deduct points from account       |       |
  |                |                   |—Reduce stock if limited          |       |
  |                |                   |—Recalculate Tier  |              |       |
  |                |                   |—UpdateAccount———▶               |       |
  |                |                   |—AddRedemption———▶               |       |
  |◀—RedemptionDto|                   |                     |           |       |
```

5.4 Award Points (Event Consumer) Flow

RewardsService — Award Points via RabbitMQ Consumer

```
WalletService                 RabbitMQ             RewardsService Consumer
      |                          |                        |
      |—Publish WalletTopUpCompletedEvent——▶|               |
      |                          |◀—consume——————————|
      |                          |                        |
      |                          |              —GetAccountByAuthUserId
      |                          |              —If null: CreateAccount()
      |                          |              —Convert topup amount → points
      |                          |              —Add points to TotalPoints
      |                          |              —Recalculate Tier
      |                          |              —UpdateAccount()
      |                          |              —Publish PointsAwardedEvent——▶
```

6. NotificationService — Detailed Design

6.1 Class Diagram

NotificationService — Class Diagram

```
┌──────────────────────────────────────────────────────────────┐
│  NotificationController                                       │
│  + GetNotifications() : IActionResult [JWT]                   │
│  + MarkAsRead(Guid id) : IActionResult [JWT]                  │
└──────────────────────────┬───────────────────────────────────┘
                           │
┌──────────────────────────▼───────────────────────────────────┐
│  INotificationService / NotificationService                   │
│  + GetNotificationsAsync(Guid authUserId) : List<NotifDto>    │
│  + MarkAsReadAsync(Guid id, Guid authUserId) : void           │
│  + CreateNotificationAsync(CreateNotifDto) : void             │
└──────────────────────────┬───────────────────────────────────┘
                           │
┌──────────────────────────▼───────────────────────────────────┐
│  INotificationRepository / NotificationRepository             │
│  + GetByAuthUserIdAsync(Guid) : List<Notification>            │
│  + GetByIdAsync(Guid) : Notification                          │
│  + CreateAsync(Notification) : void                           │
│  + UpdateAsync(Notification) : void                           │
└──────────────────────────┬───────────────────────────────────┘
                           │
┌──────────────────────────▼───────────────────────────────────┐
│  NotificationDbContext                                        │
│  DbSet<Notification> Notifications                            │
└──────────────────────────────────────────────────────────────┘


  RabbitMQ Consumers:
    WalletTopUpConsumer    → reacts to WalletTopUpCompletedEvent
    WalletTransferConsumer → reacts to WalletTransferCompletedEvent
    PointsAwardedConsumer  → reacts to PointsAwardedEvent
    KycStatusConsumer      → reacts to KycStatusChangedEvent
    WelcomeEmailConsumer   → reacts to WelcomeEmailRequestedEvent
    OtpRequestedConsumer   → reacts to OtpRequestedEvent
```

6.2 NotificationDB — ER Diagram

NotificationDB — Entity Relationship

```
┌──────────────────────────────────────────────────────────┐
│  NOTIFICATIONS                                            │
│  Id          UNIQUEIDENTIFIER  PK  DEFAULT NEWID()        │
│  AuthUserId  UNIQUEIDENTIFIER  NOT NULL                   │
│  Title       NVARCHAR(200)     NOT NULL                   │
│  Message     NVARCHAR(1000)    NOT NULL                   │
│  IsRead      BIT               DEFAULT 0                 │
│  CreatedAt   DATETIME          DEFAULT GETDATE()          │
└──────────────────────────────────────────────────────────┘


Indexes:
  IX_Notifications_AuthUserId  (for fast per-user retrieval)

Logical reference:
  AuthUserId → (logical) AuthDB.Users.Id
```

6.3 Notification Flow Sequence Diagram

NotificationService — Create & Retrieve Flow

```
System Event        RabbitMQ        NotifConsumer       NotifService     NotifRepo
    |                  |                |                    |               |
    |─Publish Event───▶|                |                    |               |
    |                  |◀─consume───────|                    |               |
    |                  |                |─CreateNotificationAsync▶           |
    |                  |                |                    |─CreateAsync──▶|
    |                  |                |                    |               |
User          NotifController   NotifService          NotifRepo
  |                |                |                    |
  |─GET /notif────▶|                |                    |
  |                |─GetNotificationsAsync(userId)▶        |
  |                |                |─GetByAuthUserId────▶
  |◀─List<NotifDto>|                |                    |
  |                |                |                    |
  |─PUT /{id}/read▶|                |                    |
  |                |─MarkAsReadAsync(id, userId)▶          |
  |                |                |─GetById────────────▶
  |                |                |─Validate ownership   |
  |                |                |─Set IsRead=true       |
  |                |                |─UpdateAsync────────▶
  |◀─200 OK────────|                |                    |
```

7. AdminService — Detailed Design

7.1 Class Diagram

AdminService — Class Diagram

```
┌──────────────────────────────────────────────────────────────┐
│  AdminController                                              │
│  + GetPendingKyc() : IActionResult [Admin JWT]                │
│  + ReviewKyc(Guid kycId, ReviewKycDto) [Admin JWT]            │
│  + GetAllUsers() : IActionResult [Admin JWT]                  │
│  + ToggleUserStatus(Guid userId) [Admin JWT]                  │
│  + GetDashboard() : IActionResult [Admin JWT]                 │
└──────────────────────────┬───────────────────────────────────┘
                           │
┌──────────────────────────▼───────────────────────────────────┐
│  IAdminService / AdminService                                 │
│  + GetPendingKycAsync() : List<KycDto>                        │
│  + ReviewKycAsync(Guid kycId, ReviewKycDto, Guid adminId)     │
│  + GetAllUsersAsync() : List<UserDto>                         │
│  + ToggleUserStatusAsync(Guid userId, Guid adminId)           │
│  + GetDashboardAsync() : DashboardDto                         │
└──────────┬──────────────────────────────┬────────────────────┘
           │                              │
┌──────────▼──────────┐        ┌──────────▼──────────────────┐
│  IAdminRepository   │        │  Typed HttpClients          │
│  + LogActionAsync   │        │  UserServiceClient          │
│    (AdminAction)    │        │    + GetPendingKyc()        │
└─────────────────────┘        │    + ReviewKyc()            │
                               │  AuthServiceClient          │
┌──────────────────────────┐   │    + GetAllUsers()          │
│  AdminDbContext          │   │    + ToggleUserStatus()     │
│  DbSet<AdminAction>      │   │    + GetEmailByAuthUserId() │
└──────────────────────────┘   └─────────────────────────────┘

  AuthTokenHandler (DelegatingHandler): Forwards Bearer JWT on outgoing calls
```

7.2 AdminDB — ER Diagram

AdminDB — Entity Relationship

```
┌──────────────────────────────────────────────────────────┐
│  ADMINACTIONS                                             │
│  Id                UNIQUEIDENTIFIER  PK  DEFAULT NEWID()  │
│  AdminAuthUserId   UNIQUEIDENTIFIER  NOT NULL             │
│  ActionType        NVARCHAR(100)     NOT NULL             │
│  TargetUserId      UNIQUEIDENTIFIER  NULL                 │
│  Remarks           NVARCHAR(500)     NULL                 │
│  PerformedAt       DATETIME          DEFAULT GETDATE()    │
└──────────────────────────────────────────────────────────┘


ActionType values (as used in codebase):
  KYC_APPROVED  |  KYC_REJECTED  |  USER_TOGGLED

Logical references:
  AdminAuthUserId → (logical) AuthDB.Users.Id
  TargetUserId    → (logical) AuthDB.Users.Id
```

7.3 Admin KYC Review Sequence Diagram

AdminService — KYC Review Flow

```
Admin    AdminController   AdminService   UserServiceClient  AuthTokenHandler
UserService
  |           |               |                 |                  |
|
  |─PUT review▶|              |                 |                  |
|
  |           |─ReviewKycAsync▶                 |                  |
|
  |           |               |─Call UserServiceClient.ReviewKyc(kycId, dto)
  |           |               |                 |─Attach Bearer JWT──────────────▶|
  |           |               |                 |                  |
|
  |           |               |                 |
UserService.ReviewKyc()
  |           |               |◀─200 OK─────────|                  |
|
  |           |               |─Log AdminAction(KYC_APPROVED/REJECTED, adminId)
  |           |               |─AdminRepo.LogActionAsync()          |
|
  |◀─200 OK───|               |                 |                  |
|
```

7.4 Dashboard Assembly Flow

AdminService — Dashboard Flow

```
Admin    AdminController   AdminService   AuthServiceClient  UserServiceClient
  |          |               |                |                  |
  |—GET /dashboard▶|         |                |                  |
  |          |—GetDashboardAsync▶             |                  |
  |          |               |—GetAllUses()—▶|                  |
  |          |               |◀—List<UserDto>—|                  |
  |          |               |—GetPendingKyc()———————————————▶|
  |          |               |◀—List<KycDto>———————————————|
  |          |               |—Assemble DashboardDto              |
  |          |               |  (total users, pending KYC count)  |
  |◀—DashboardDto—|          |                |                  |
```

8. Exception Handling Design

8.1 Global Exception Middleware

Each service implements a global exception handling middleware that catches unhandled exceptions and returns a standardized error response. The middleware logs the exception and returns HTTP 500 with a generic error message to avoid leaking stack traces.

8.2 Exception Categories

Exception Type	HTTP Status	Example Scenario
Validation failure (FluentValidation / DataAnnotations)	400 Bad Request	Missing required DTO field, invalid email format
Business rule violation (duplicate email, insufficient balance)	400 / 409 Conflict	Wallet already exists, insufficient funds for transfer
Authentication failure (invalid JWT)	401 Unauthorized	Expired or missing JWT
Authorization failure (wrong role)	403 Forbidden	Regular user accessing admin endpoint
Resource not found	404 Not Found	Wallet not found for given AuthUserId
Unhandled server exception	500 Internal Server Error	Database connection failure

9. Caching Design

Service	Cache Key Pattern	Cached Value	TTL / Eviction
AuthService	otp:{email}	OTP string	10 minutes absolute
AuthService	otp_verified:{email}	bool flag	Short (registration window)
AuthService	refresh:{userId}	Refresh token string	Configurable (hours/days)
AuthService	reset_otp:{email}	Reset OTP string	10 minutes absolute
WalletService	wallet_balance:{authUserId}	Balance decimal	Short duration; cleared on topup/transfer
RewardsService	reward_catalog	List<RewardCatalog>	5 minutes sliding

All caching uses ASP.NET Core IMemoryCache (in-process). This means cache data is not shared across service instances and is lost on process restart. For production high-availability deployments, Redis is the recommended replacement.

10. Messaging Design

10.1 Event Publishers

Service	Event Name	Trigger	Payload Fields
AuthService	OtpRequestedEvent	SendOtp called	Email, OtpCode
AuthService	WelcomeEmailRequestedEvent	User registered / Google login	Email, FullName (if available)
WalletService	WalletTopUpCompletedEvent	TopUp successful	AuthUserId, Amount, WalletId
WalletService	WalletTransferCompletedEvent	Transfer successful	SenderAuthUserId, ReceiverAuthUserId, Amount
RewardsService	PointsAwardedEvent	Points awarded to account	AuthUserId, PointsAwarded, NewTotal
AdminService	KycStatusChangedEvent	KYC reviewed	AuthUserId, Status, Remarks

10.2 Event Consumers

Consumer Service	Consumed Event	Action Taken
NotificationService	WalletTopUpCompletedEvent	Create 'Top-Up Successful' notification
NotificationService	WalletTransferCompletedEvent	Create 'Transfer Completed' notification for both parties
NotificationService	PointsAwardedEvent	Create 'Points Awarded' notification
NotificationService	KycStatusChangedEvent	Create 'KYC Status Updated' notification
NotificationService	WelcomeEmailRequestedEvent	Create welcome notification
NotificationService	OtpRequestedEvent	Trigger OTP delivery (email simulation)
RewardsService	WalletTopUpCompletedEvent	Award points based on top-up amount

11. Security Design

11.1 JWT Claims

Claim	Value	Used By
NameIdentifier	AuthUserId (GUID)	All services to identify the caller
Email	User email (lowercase)	Informational, WalletService stores it
Role	User or Admin	Role-based authorization checks

11.2 Role Checks

- [Authorize] — requires valid JWT (any role)
- [Authorize(Roles = "Admin")] — requires Admin role claim
- AdminService applies [Authorize(Roles = "Admin")] to all controllers/actions
- Internal endpoints (e.g., /api/auth/users/{id}/email) are called service-to-service; HTTPS and API gateway configuration limit exposure

11.3 Token Forwarding in AdminService

AdminService registers AuthTokenHandler as a DelegatingHandler on its typed HttpClients. On each outgoing request, the handler reads the Authorization header from the current IHttpContextAccessor and attaches it to the downstream request, preserving admin identity for authorization checks in UserService and AuthService.

12. Testing Scope

12.1 Unit Test Areas

- AuthService: BCrypt hash/verify logic, JWT generation, OTP generation and caching, registration validations
- WalletService: balance calculation, insufficient-funds check, cache eviction on mutation
- RewardsService: tier calculation logic, point deduction, stock validation
- NotificationService: ownership check on mark-as-read
- AdminService: action logging, token forwarding handler

12.2 Integration Test Areas

- AuthService: full registration → OTP → verify → register flow against test database
- WalletService: create → topup → transfer flow, verifying ledger entries and balance
- RewardsService: event consumer consuming WalletTopUpCompletedEvent and awarding points
- AdminService: KYC review flow calling UserService via HttpClient

12.3 Critical Test Cases

Test Case	Service	Expected Outcome
Registration with duplicate email	AuthService	409 Conflict or 400
Registration without OTP verification	AuthService	400 Bad Request
Login with inactive account	AuthService	401 Unauthorized
Transfer with insufficient balance	WalletService	400 Bad Request
Transfer to own wallet	WalletService	400 Bad Request
Redeem with insufficient points	RewardsService	400 Bad Request
Redeem out-of-stock catalog item	RewardsService	400 Bad Request
Mark notification as read by non-owner	NotificationService	403 Forbidden
Admin endpoint with User role JWT	AdminService	403 Forbidden
KYC submission duplicate	UserService	409 Conflict or 400

13. Complete Business Workflow Diagrams

13.1 End-to-End User Onboarding Workflow

Full User Onboarding Flow (Activity Diagram)

```
START
  │
  ▼
[1] POST /auth/send-otp {email}
  │   → OTP generated + cached (10 min)
  │   → OtpRequestedEvent → NotificationService
  ▼
[2] POST /auth/verify-otp {email, otp}
  │   → OTP validated from cache
  │   → Email marked verified in cache
  ▼
[3] POST /auth/register {email, phone, password, ...}
  │   → Check email verified flag
  │   → Check unique email + phone
  │   → BCrypt hash password
  │   → Save User (Role=User, IsActive=true)
  │   → Publish WelcomeEmailRequestedEvent
  │   → Return JWT + RefreshToken
  ▼
[4] POST /user/profile {fullName, dob, address, ...}  [JWT]
  │   → AuthUserId from JWT
  │   → Check no existing profile
  │   → Save UserProfile
  ▼
[5] POST /user/kyc {documentType, documentNumber}  [JWT]
  │   → Check no existing KYC
  │   → Save KycSubmission (Status=Pending)
  ▼
[6] POST /wallet/create  [JWT]
  │   → Check no existing wallet
  │   → Create Wallet (Balance=0, IsActive=true)
  ▼
END → User fully onboarded
```

13.2 Wallet Top-Up and Reward Award Combined Flow

WalletService → RewardsService → NotificationService (Full Event Chain)

```
User     WalletService      RabbitMQ     RewardsService   NotificationService
 |            |                |               |                  |
 |—POST topup▶|                |               |                  |
 |            |—Create CREDIT ledger entry     |                  |
 |            |—Update balance                 |                  |
 |            |—Publish WalletTopUpCompletedEvent▶               |
 |            |                |◀—consume———|                  |
 |            |                |               |—AwardPointsAsync |
 |            |                |               |—Publish PointsAwardedEvent▶|
 |            |                |◀—consume————————————————————|
 |            |                |               |      —CreateNotification (Points
Awarded)
 |            |◀—consume————————————————————————————————————|
 |            |(topup event also consumed by NotificationService) |
 |            |               TopUp notification also created———▶
 |◀—TransactionDto|            |               |                  |
```

— End of Low-Level Design Document —